

Arreglo Unidimensional

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Arreglo Unidimensional

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

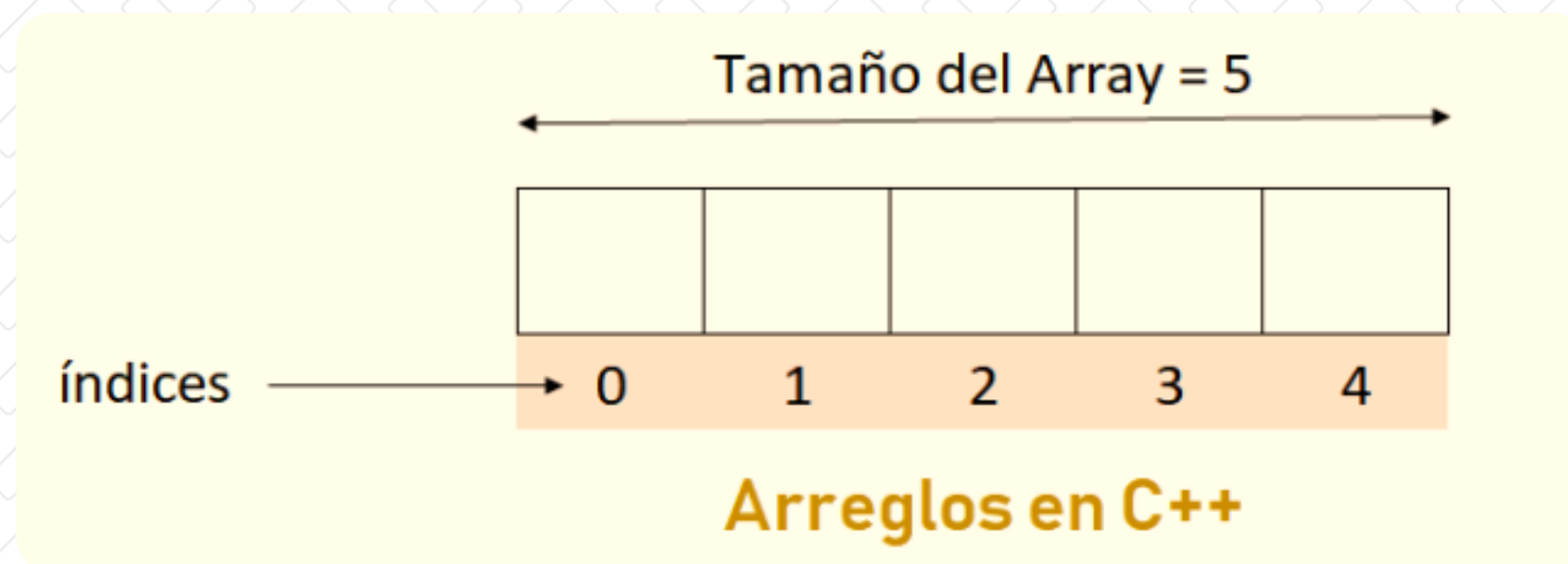
Comprender el concepto de arreglos en programación, utilizando sus diferentes operaciones para almacenar, manipular y procesar datos de manera eficiente en diversos lenguajes de programación.

Objetivos Específicos

- Identificar la estructura y características de los arreglos, diferenciando sus tipos (unidimensionales, multidimensionales, indeterminados) y su uso en distintos lenguajes de programación.
- Implementar operaciones básicas y avanzadas en arreglos, para resolver problemas de manera eficiente.

¿Qué es un arreglo?

Un arreglo es como una fila de cajas en la memoria, donde cada caja puede guardar un valor del mismo tipo (por ejemplo, solo números enteros o solo letras). Cada caja tiene una posición identificada por un número, que se llama índice, y la primera siempre empieza en 0.



Tipos de arreglos

Unidimensionales

Bidimensionales

Multidimensionales

Indeterminados

Recorrido de arreglos unidimensionales

- El **recorrido de un arreglo unidimensional** significa acceder a cada uno de sus elementos, ya sea por tres motivos para mostrarlos, modificarlos o hacer cálculos con ellos.
- Para realizar el recorrido se puede hacer de varias formas, dependiendo de lo que se vaya a realizar:

Recorrido con for (índices)

Recorrido con while
(usando un índice)

Recorrido con punteros

Característica del arreglo unidimensional

Para saber qué tipo de arreglo es, utilizamos las características para poder identificarlos las características para los arreglos unidimensionales.

-  Almacenan datos del mismo tipo (int, float, char, etc).
-  Cada elemento se identifica con un índice que empieza desde 0.
-  Su tamaño es fijo, no se puede cambiar en tiempo de ejecución.
-  Ocupan espacio contiguo en memoria, lo que permite acceso rápido a los elementos.

Comprendiendo los arreglos unidimensionales

Un arreglo unidimensional es como una lista de valores del mismo tipo (todos números, o todas letras, por ejemplo). Cuando lo declaras, tienes que decirle al programa cuántos elementos vas a guardar, y esta reserva espacio en la memoria para esa cantidad.

Se usan cuando quieres almacenar varios datos similares sin crear una variable para cada uno.

Por ejemplo, si quieres guardar las calificaciones de 5 estudiantes, en vez de usar 5 variables (nota1, nota2, etc.), usas un arreglo

Para declarar un arreglo de una sola dimensión se usa el siguiente formato:

Tipo de dato + variable[tamaño];

“El ciclo for nos ayuda a recorrer todo el arreglo sin escribir el mismo código muchas veces. Solo le decimos: ‘Haz esto desde la posición 0 hasta la última’ y él lo hace solo.”

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Declaramos un arreglo de 5 enteros
6      int numeros[5] = {10, 20, 30, 40, 50};
7
8      // Imprimimos cada elemento del arreglo
9      for (int i = 0; i < 5; i++) {
10         cout << "Elemento en la posición " << i << " = " << numeros[i] << endl;
11     }
12
13     return 0;
14 }
```

Output

```
Elemento en la posición 0 = 10
Elemento en la posición 1 = 20
Elemento en la posición 2 = 30
Elemento en la posición 3 = 40
Elemento en la posición 4 = 50
```

`int numeros[5] = {10, 20, 30, 40, 50};`

- Tipo de dato
- Variable
- Tamaño

Valores en
cada casilla

“La computadora empieza a contar desde 0, no desde 1. Por eso, la primera casilla del arreglo se llama posición 0.”

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Comprendiendo los arreglos unidimensionales

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int tamano;
6
7      // Solicitar al usuario el tamaño del arreglo entre 3 y 6
8      cout << "Ingrese el tamaño del arreglo (mínimo 3, máximo 6): ";
9      cin >> tamano;
10
11     // Validar que el tamaño esté en el rango permitido
12     if (tamano < 3 || tamano > 6) {
13         cout << "Error: El tamaño debe estar entre 3 y 6." << endl;
14         return 0; // Finaliza el programa si el tamaño no es válido
15     }
16
17     // Declarar el arreglo con el tamaño dado por el usuario
18     int numeros[tamano];
19
20     // Pedir al usuario que ingrese los valores uno por uno
21     for (int i = 0; i < tamano; i++) {
22         cout << "Ingrese un número para la posición [" << i << "]: ";
23         cin >> numeros[i]; // Guarda el valor en la posición correspondiente
24     }
25
26     // Mostrar todos los valores ingresados
27     cout << "\nValores almacenados en el arreglo:" << endl;
28     for (int i = 0; i < tamano; i++) {
29         cout << "Posición [" << i << "]: " << numeros[i] << endl;
30     }
31
32     return 0;
```

`int numeros[tamano];`

- Tipo de dato
- Variable
- Tamaño

`cin >> numeros[i]`

Guarda el valor
en la posición
de cada
casilla.

- Cómo crear arreglos con tamaño dinámico controlado por el usuario.
- Cómo recorrer un arreglo para ingresar y mostrar datos.
- Cómo validar rangos de entrada para evitar errores.

“El ciclo for nos ayuda a recorrer todo el arreglo sin escribir el mismo código muchas veces. Solo le decimos: ‘Haz esto desde la posición 0 hasta la última’ y él lo hace solo.”

🧠 Explicación del código (en lenguaje sencillo)

1. 🚩 Se le pide al usuario que elija cuántos elementos quiere (entre 3 y 6).
2. ✅ Se valida que esté en ese rango. Si no lo está, se muestra un error y se termina el programa.
3. 🧠 Si el valor es válido, se crea el arreglo con ese tamaño exacto.
4. 🎯 Se usa un ciclo `for` para pedir un número por cada posición del arreglo.
5. 🖨️ Luego, se imprime lo que el usuario ingresó, mostrando el índice y el valor de cada elemento.

Output

```
Ingrese el tamaño del arreglo (mínimo 3, máximo 6): 4
Ingrese un número para la posición [0]: 2
Ingrese un número para la posición [1]: 5
Ingrese un número para la posición [2]: 22
Ingrese un número para la posición [3]: 9
```

```
Valores almacenados en el arreglo:
Posición [0]: 2
Posición [1]: 5
Posición [2]: 22
Posición [3]: 9
```


Arreglo Unidimensional

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Arreglo Unidimensional

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

